

## **ЛАБОРАТОРНАЯ РАБОТА №6**

### **Мандатное разграничение прав в Linux**

Отчёт по выполнению лабораторной работы

Дисциплина: Информационная безопасность компьютерных сетей

Тема: Мандатное разграничение прав в Linux. SELinux и веб-сервер Apache

Студент: Оганнисян Гор 1132243806

Группа: НБИбд-01-24

Преподаватель: Кулябов Дмитрий Сергеевич

Дата: 01.05.2026

## **1. Цель работы**

Развитие навыков администрирования операционной системы Linux, получение практического знакомства с технологией SELinux и проверка работы мандатного разграничения прав совместно с веб-сервером Apache. В ходе работы исследуется влияние контекстов безопасности SELinux на доступ веб-сервера к файлам и возможность запуска Apache на нестандартном TCP-порту.

## **2. Краткие теоретические сведения**

SELinux — это механизм мандатного разграничения доступа в Linux. В отличие от обычной дискреционной модели прав, где доступ определяется владельцем файла и битами чтения, записи и выполнения, SELinux дополнительно проверяет контекст безопасности субъекта и объекта. Поэтому даже при разрешающих обычных правах доступ может быть запрещён политикой SELinux.

Контекст SELinux обычно имеет вид `user:role:type:level`. Для данной лабораторной работы наиболее важна часть `type`. Процесс веб-сервера Apache обычно выполняется в домене `httpd_t`, а файлы веб-каталога должны иметь тип `httpd_sys_content_t`. Если файл получит другой тип, например `samba_share_t`, процесс `httpd` может не получить к нему доступ, даже если обычные права Linux разрешают чтение.

В режиме `enforcing` политика SELinux реально применяется и запрещает недопустимые действия. В режиме `permissive` нарушения только фиксируются в журналах, но не блокируются. В лабораторной работе используется режим `enforcing` и политика `targeted`, так как они позволяют наглядно показать работу мандатной защиты.

Кроме файловых контекстов, SELinux контролирует сетевые порты. Apache разрешено прослушивать только порты, помеченные типом `http_port_t`. Поэтому перенос сервера с порта 80 на порт 81 без добавления соответствующего правила приводит к ошибке запуска.

## **3. Ход выполнения работы**

### **3.1. Проверка режима SELinux и используемой политики**

Сначала проверяется, что SELinux работает в режиме `enforcing` и используется политика `targeted`.

### **Команды:**

```
getenforce  
sestatus
```

**Результат:** Команда `getenforce` должна вывести `Enforcing`. В выводе `sestatus` необходимо проверить строки `Current mode: enforcing` и `Loaded policy name: targeted`. Это означает, что SELinux не просто фиксирует нарушения, а реально блокирует действия, запрещённые политикой.

## **3.2. Проверка работы веб-сервера Apache**

Далее проверяется состояние службы Apache. Если веб-сервер не запущен, его необходимо запустить от имени суперпользователя.

### **Команды:**

```
su -  
service httpd status  
service httpd start  
service httpd status  
exit
```

**Результат:** После запуска служба `httpd` должна перейти в состояние `running`. При обращении к локальному веб-серверу через браузер или консольную программу должна открываться стандартная страница Apache либо иной ответ сервера.

## **3.3. Определение контекста безопасности процесса Apache**

После запуска Apache определяется SELinux-контекст процессов `httpd`.

### **Команды:**

```
ps auxZ | grep httpd  
ps -eZ | grep httpd
```

**Результат:** В выводе команды у процессов Apache должен отображаться контекст вида `system_u:system_r:httpd_t:s0`. Тип `httpd_t` показывает, что процесс относится к домену веб-сервера и подчиняется правилам SELinux для Apache.

## **3.4. Просмотр переключателей SELinux и статистики политики**

Для Apache просматриваются SELinux-переключатели, а также общая информация о политике, пользователях, ролях и типах.

### **Команды:**

```
sestatus -b | grep httpd
seinfo
seinfo -u
seinfo -r
seinfo -t | head
```

**Результат:** Большая часть переключателей Apache находится в положении off. Это нормальное состояние для политики targeted: дополнительные возможности веб-сервера отключены по умолчанию и включаются только при необходимости. Команда seinfo показывает количество правил, пользователей, ролей и типов, определённых в политике SELinux.

### 3.5. Проверка контекстов каталогов /var/www и /var/www/html

Проверяется тип файлов и подкаталогов, расположенных в стандартном каталоге веб-сервера.

#### Команды:

```
ls -lZ /var/www
ls -lZ /var/www/html
ls -ld /var/www/html
```

**Результат:** Для каталогов веб-сервера обычно используется тип httpd\_sys\_content\_t. По обычным правам Linux каталог /var/www/html принадлежит root, поэтому создавать файлы в нём может суперпользователь. Это подтверждает, что для записи в каталог требуются административные права.

### 3.6. Создание тестового HTML-файла

От имени суперпользователя создаётся файл /var/www/html/test.html с простым HTML-содержимым.

#### Команды:

```
su -
cat > /var/www/html/test.html <<'EOF'
<html>
<body>test</body>
</html>
EOF
ls -Z /var/www/html/test.html
exit
```

**Результат:** Созданному файлу по умолчанию должен быть назначен контекст, подходящий для веб-сервера, например unconfined\_u:object\_r:httpd\_sys\_content\_t:s0 или system\_u:object\_r:httpd\_sys\_content\_t:s0. Главная часть контекста здесь — тип httpd\_sys\_content\_t.

### 3.7. Проверка доступа к файлу через веб-сервер

После создания файла выполняется обращение к нему через Apache.

#### Команды:

```
curl http://127.0.0.1/test.html
```

**Результат:** Файл должен успешно открыться. В ответе выводится HTML-код с текстом test. Это означает, что обычные права Linux и контекст SELinux позволяют процессу httpd\_t читать файл типа httpd\_sys\_content\_t.

### 3.8. Сопоставление контекста файла с документацией httpd\_selinux

Изучается справка по SELinux-политике Apache и повторно проверяется контекст файла test.html.

#### Команды:

```
man httpd_selinux  
ls -Z /var/www/html/test.html
```

**Результат:** Справка показывает, какие типы файлов предназначены для работы Apache. Тип httpd\_sys\_content\_t используется для статического содержимого веб-сервера. Именно поэтому файл test.html при таком контексте успешно открывается через браузер.

### 3.9. Изменение контекста файла на samba\_share\_t и повторная проверка доступа

Для демонстрации мандатного запрета тип файла меняется на такой, который не предназначен для чтения Apache.

#### Команды:

```
su -  
chcon -t samba_share_t /var/www/html/test.html  
ls -Z /var/www/html/test.html  
exit  
curl -i http://127.0.0.1/test.html  
ls -l /var/www/html/test.html
```

**Результат:** После изменения типа на samba\_share\_t обращение к файлу через веб-сервер должно завершиться ошибкой Forbidden. При этом обычные права файла могут разрешать чтение всем пользователям, например -rw-r--r--, но SELinux всё равно запрещает доступ процессу httpd\_t к объекту с неподходящим типом.

### 3.10. Анализ журналов после отказа доступа

После получения ошибки Forbidden просматриваются журналы Apache и системные журналы SELinux.

#### Команды:

```
tail -n 20 /var/log/messages  
tail -n 20 /var/log/httpd/error_log  
tail -n 20 /var/log/audit/audit.log
```

**Результат:** В журналах должны появиться сообщения об отказе в доступе. В audit.log обычно фиксируется AVC denied, где видно, что процесс httpd пытался прочитать файл с неподходящим SELinux-типом. Это подтверждает, что причина отказа связана не с обычными правами доступа, а с политикой SELinux.

### 3.11. Перевод Apache на нестандартный порт 81

Далее проверяется, разрешит ли SELinux веб-серверу прослушивать порт 81. Для этого в конфигурационном файле Apache строка Listen 80 заменяется на Listen 81.

#### Команды:

```
su -  
cp /etc/httpd/conf/httpd.conf /etc/httpd/conf/httpd.conf.bak  
sed -i 's/^Listen 80/Listen 81/' /etc/httpd/conf/httpd.conf  
service httpd restart
```

**Результат:** Перезапуск веб-сервера должен завершиться ошибкой. Причина состоит в том, что порт 81 по умолчанию не имеет типа http\_port\_t, а политика SELinux не разрешает процессу httpd\_t привязываться к этому порту.

### 3.12. Анализ журналов после ошибки запуска Apache

Чтобы подтвердить причину сбоя, просматриваются системные журналы и журналы Apache.

#### Команды:

```
tail -n 20 /var/log/messages  
tail -n 20 /var/log/httpd/error_log  
tail -n 20 /var/log/httpd/access_log  
tail -n 20 /var/log/audit/audit.log
```

**Результат:** В журналах могут появиться записи о невозможности выполнить операцию name\_bind для порта 81. Это означает, что процесс Apache не получил разрешение SELinux на прослушивание указанного TCP-порта.

### 3.13. Добавление порта 81 к типу http\_port\_t

Порт 81 добавляется в список портов, разрешённых для веб-сервера Apache.

#### Команды:

```
semanage port -a -t http_port_t -p tcp 81
semanage port -l | grep http_port_t
service httpd restart
```

**Результат:** После добавления правила порт 81 появляется в списке http\_port\_t. Повторный запуск Apache проходит успешно, потому что SELinux теперь считает порт 81 допустимым для процесса httpd\_t.

### 3.14. Возврат правильного контекста файлу и проверка доступа через порт 81

Контекст test.html возвращается к типу httpd\_sys\_content\_t, после чего выполняется обращение к файлу через нестандартный порт.

#### Команды:

```
chcon -t httpd_sys_content_t /var/www/html/test.html
ls -Z /var/www/html/test.html
curl http://127.0.0.1:81/test.html
```

**Результат:** После восстановления контекста файл снова доступен через веб-сервер. В ответе должна отображаться страница с содержимым test. Это показывает, что для успешной работы должны одновременно выполняться два условия: файл имеет разрешённый тип, а порт относится к http\_port\_t.

### 3.15. Возврат исходных настроек и удаление тестовых данных

После завершения эксперимента конфигурация Apache возвращается к порту 80, правило для порта 81 удаляется, а тестовый файл очищается.

#### Команды:

```
sed -i 's/^Listen 81/Listen 80/' /etc/httpd/conf/httpd.conf
service httpd restart
semanage port -d -t http_port_t -p tcp 81
semanage port -l | grep http_port_t
rm /var/www/html/test.html
exit
```

**Результат:** Служба Apache снова работает на стандартном порту 80. Порт 81 удалён из списка http\_port\_t, а файл test.html удалён из каталога веб-сервера. Лабораторный стенд возвращён в исходное состояние.

## 4. Таблица 4.1. Проверка работы SELinux и Apache

Ниже приведены основные результаты экспериментов, связанных с контекстами файлов, процессом Apache и нестандартным портом 81.

| Операция                               | Результат | Пояснение                                                                                |
|----------------------------------------|-----------|------------------------------------------------------------------------------------------|
| Проверка getenforce и sestatus         | Да        | SELinux работает в режиме enforcing, используется политика targeted.                     |
| Проверка процесса Apache               | Да        | Процесс httpd имеет SELinux-домен httpd_t.                                               |
| Создание test.html в /var/www/html     | Да        | Файл получает тип httpd_sys_content_t, подходящий для статического веб-содержимого.      |
| Открытие test.html через порт 80       | Да        | Apache может читать файл с типом httpd_sys_content_t.                                    |
| Смена типа файла на samba_share_t      | Да        | Контекст файла изменяется командой chcon.                                                |
| Открытие файла с типом samba_share_t   | Нет       | Apache получает ошибку Forbidden, так как SELinux запрещает доступ к неподходящему типу. |
| Запуск Apache на порту 81 без semanage | Нет       | SELinux запрещает процессу httpd_t привязку к порту, не помеченному как http_port_t.     |
| Добавление порта 81 к http_port_t      | Да        | После semanage port -a Apache успешно запускается на порту 81.                           |
| Возврат контекста httpd_sys_content_t  | Да        | Файл снова становится доступен через веб-сервер.                                         |
| Удаление правила порта 81 и test.html  | Да        | Стенд возвращается в исходное состояние.                                                 |

## 5. Таблица 5.1. Причины отказов и способы исправления

В ходе работы возникали два основных отказа: запрет чтения файла и запрет запуска Apache на нестандартном порту.

| Ситуация                               | Видимый результат             | Причина                                                    | Исправление                                                                    |
|----------------------------------------|-------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------|
| Файл test.html имеет тип samba_share_t | Forbidden в браузере или curl | Домен httpd_t не имеет разрешения читать файлы этого типа. | Вернуть тип: <code>chcon -t httpd_sys_content_t /var/www/html/test.html</code> |



|                                                 |                             |                                          |                                                            |
|-------------------------------------------------|-----------------------------|------------------------------------------|------------------------------------------------------------|
| Apache переведён на порт 81                     | Служба httpd не запускается | Порт 81 не относится к типу http_port_t. | Добавить порт: semanage port -a -t http_port_t -p tcp 81   |
| После завершения работы порт 81 больше не нужен | Лишнее правило в SELinux    | Изменения стенда должны быть возвращены. | Удалить правило: semanage port -d -t http_port_t -p tcp 81 |

## 6. Анализ результатов

Эксперимент показал, что SELinux выполняет независимую проверку доступа, которая не сводится к обычным правам Linux. Если файл `/var/www/html/test.html` имеет права на чтение для всех пользователей, это ещё не означает, что процесс Apache сможет его прочитать. Для успешного доступа важен SELinux-тип файла. При типе `httpd_sys_content_t` доступ разрешён, а при типе `samba_share_t` веб-сервер получает отказ.

Второй важный результат связан с сетевыми портами. Apache по умолчанию работает на порту 80, который уже разрешён политикой SELinux для веб-сервера. При попытке перенести Apache на порт 81 возникает ошибка, потому что этот порт не входит в набор портов типа `http_port_t`. После добавления порта командой `semanage port -a -t http_port_t -p tcp 81` запуск становится возможным.

Таким образом, SELinux контролирует не только доступ к файлам, но и сетевые действия процессов. Это повышает безопасность системы: даже если обычные права Linux настроены недостаточно строго или администратор случайно изменил файл, политика SELinux может дополнительно ограничить потенциально опасные действия.

## 7. Вывод

В ходе лабораторной работы были изучены основы мандатного разграничения прав в Linux с использованием SELinux. Были проверены режим `enforcing` и политика `targeted`, определён контекст безопасности процесса Apache, изучены контексты каталогов `/var/www` и `/var/www/html`, создан тестовый HTML-файл и проверен доступ к нему через веб-сервер.

Практически подтверждено, что изменение SELinux-типа файла с `httpd_sys_content_t` на `samba_share_t` приводит к запрету доступа через Apache, несмотря на разрешающие обычные права Linux. Также установлено, что запуск Apache на нестандартном порту 81 невозможен без добавления этого

порта к типу `http_port_t`. Работа показала, что SELinux является важным механизмом дополнительной защиты, который контролирует действия процессов на уровне политики безопасности.

## 8. Приложения

В этот раздел следует вставить снимки экрана выполнения основных команд и полученных результатов:

- `getenforce` и `sestatus`;
- `service httpd status` и `service httpd start`;
- `ps auxZ | grep httpd` или `ps -eZ | grep httpd`;
- `sestatus -b | grep httpd` и `seinfo`;
- `ls -lZ /var/www` и `ls -lZ /var/www/html`;
- создание `/var/www/html/test.html` и вывод `ls -Z`;
- успешное открытие `http://127.0.0.1/test.html`;
- изменение контекста `chcon -t samba_share_t` и ошибка `Forbidden`;
- журналы `/var/log/messages`, `/var/log/httpd/error_log` и `/var/log/audit/audit.log`;
- замена `Listen 80` на `Listen 81` и ошибка перезапуска Apache;
- команда `semanage port -a -t http_port_t -p tcp 81` и проверка списка портов;
- успешное открытие `http://127.0.0.1:81/test.html` после восстановления контекста;
- возврат `Listen 80`, удаление порта 81 и удаление `test.html`.